Tic-tac-toe projected model count benchmarks

Ivor Spence, Queen's University Belfast

March 21, 2024

Abstract

Tic-tac-toe is a two player game in which the players take turns to claim a square on a 3x3 board. The game is won if a player claims three squares in a horizontal, vertical or diagonal line. The ttt generator creates projected model counting instances to count the number of valid game states for tic-tac-toe games of different sizes.

1 Introduction

In the standard tic-tac-toe game two players X and O take turns to claim a square in a 3x3 board. By convention X goes first, and the game stops either because one player has a winning straight line of length three or because the board is full without any winning line, in which case the game is drawn (see Figure 1). For the standard game there are 5478 valid states of the board (including this initial empty board). In addition to tracing how the game is played, it is possible to determine whether a proposed state is valid by applying the following rules:

1. Each square is blank or else contains either X or O.
2. The number of Xs must be equal to the number of Os (in which case it is X's turn next) or one more than the number of Os (in which case it is O's turn).
3. If there is a winning line of Xs then X must have just played and so there must be one more X than O.
4. If there is a winning line of Os then O must have just played so the number of Xs and Os must be equal.
5. It must be possible to reach the current state without any previous winning lines. This can be expressed by saying that there must be (at least) one square which, if made blank, would result in there being no winning lines.

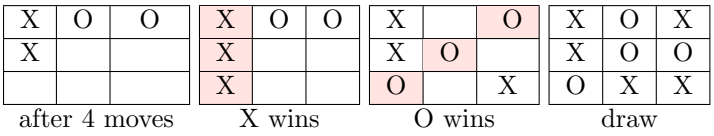


Figure 1: States for the standard game

2 Benchmarks

The standard game on a 3x3 board with winning lines of length 3 results in projected model count instances which are not challenging for modern solvers. Therefore, the generator has been configured to use arbitrary board dimensions and lengths of winning line. The sizes of some instances, together with the solving time using the `pmcLinux` solver and the expected counts, are shown in Table 1. A solving time of "unknown" means that the solver did not finish within 1 hour. The results for the standard game are highlighted in red and the suggested instances for the mc2024 competition are highlighted in green.

rows	cols	line length	vars	clauses	time(s)	expected count	
3	3	1	1025	4547	<0.1	10	
3	3	2	993	4195	<0.1	1234	
3	3	3	969	3931	<0.3	5478	standard 3x3 game
3	3	4	953	3755	<0.2	6046	
3	4	1	6727	27811	<0.1	13	
3	4	2	6688	27279	2	10562	
3	4	3	6658	26859	13	111973	
3	4	4	6636	26551	12	142231	
3	5	1	55801	224722	2	16	
3	5	2	55757	223974	95	85733	
3	5	3	55721	223362	1333	2286668	ttt-3-5-3.cnf
3	5	4	55693	222886	1646	3414564	
3	5	5	55687	222784	1694	3485916	ttt-3-5-5.cnf
4	4	1	101448	407549	8	17	
4	4	2	101404	406757	329	159765	ttt-4-4-2.cnf
4	4	3	101368	406109	unknown	6036001	ttt-4-4-3.cnf

Table 1: Results

3 Generator

The generator ttt is written in c and a Makefile is provided. It can be invoked by

```
./ttt rows columns line-length
```

and creates the files `ttt-rows-columns-length.cnf` (which is the projected model count instance) and `ttt-rows-columns-length.var` (a list of the projected variables, required by some solvers to be in a separate file). The standard board is thus represented by `ttt-3-3-3.cnf`.

A separate program `reccount` is provided, which uses a stand-alone algorithm to calculate the number of states. It uses the same arguments as `ttt`. For example,

```
./reccount 3 3 3
```

calculates that the standard tic-tac-toe game has 5478 reachable states. `reccount` requires approximately $3^{\text{rows}*\text{cols}}$ bits of memory, so rows*cols = 24, needing approximately 33GB, is likely to be the largest practical size.

These projected model count instances are proposed as challenging tests for solvers - the results are more efficiently calculated by other methods.

4 cnf file

For a board with `cells` squares, variables 1...`cells` indicate that the corresponding square contains an X, whereas variables `cells`+1...2*`cells` indicate the presence of an O. Thus in the standard board the clause -1 -10 0 indicates that the first square can't contain both an X and an O. Many additional variables and clauses are required to implement the rules given above. A given state of the board may be represented by different models, for example in the "X wins" board of Figure 1 any of the three Xs in the winning line could have been the final move. In order to ensure that each state of the board is only counted once, the projected variables are 1...2*`cells`. For example the standard game cnf file has 27440 satisfying models if projection is not used, rather then the correct count of 5478.